

Sistema Híbrido de Análise de Sentimentos em Português com Modelos Locais

Rui Pedro Gil

Universidade da Beira Interior, Departamento de Informática, Covilhã, Portugal

Abstract

A análise de sentimentos em textos apresenta desafios significativos, especialmente na interpretação de opiniões ambíguas e na transparência das previsões. Este relatório descreve o desenvolvimento de um sistema híbrido de PLN, 100% local e *offline*, que combina a previsão de modelos *encoder-only* com a interpretabilidade generativa de *Large Language Models* (LLMs). O sistema desenvolvido utiliza o XLM-RoBERTa, submetido a *fine-tuning* num *corpus* de 3000 avaliações de comércio eletrónico em português, para a classificação discriminativa do sentimento, além disso está capacitado para generalizar para outras áreas de comércio e também para opiniões e textos provenientes das redes sociais. Adicionalmente, o LLM Mistral 7B via Ollama é aqui implementado para gerar explicações em linguagem natural que justifiquem a decisão do modelo classificador. Para além disso, foi desenvolvida uma interface gráfica interativa que unifica estes dois níveis, permitindo a submissão de texto e a visualização dos resultados em tempo real. O modelo treinado foi avaliado num conjunto de teste isolado de 300 avaliações reais, alcançando uma acurácia (*accuracy*) de 78.0%. Os resultados obtidos demonstram a eficácia da adaptação de domínio e evidenciam a grande importância da justificação generativa por parte do *decoder* para mitigar a ambiguidade relativa à classe neutra.

Keywords

Análise de Sentimentos, Modelos de Linguagem, XLM-RoBERTa, Fine-Tuning, Mistral 7B, Ollama, IA Explicável

1. Introdução

A análise de sentimentos é uma das tarefas mais estudadas no âmbito do Processamento de Linguagem Natural (PLN). Modelos que se baseiam na arquitetura *Transformer* [1], como o RoBERTa [2], atingem rácios de acurácia extremamente notáveis. Contudo, estes sistemas discriminativos agem a maior parte das vezes como *black boxes* ao fornecerem uma (*label*) e uma probabilidade, mas não justificam o raciocínio subjacente que induziu o sistema a tomar a respetiva decisão, o que limita a sua adoção em diversos cenários críticos.

A maioria dos sistemas da atualidade que se focam em resolver este problema de transparência, assenta em serviços na *cloud* (APIs externas), o que suscita questões de privacidade. Este trabalho propõe uma abordagem alternativa e inovadora, um sistema híbrido que combina a precisão estatística de um modelo de classificação especializado no domínio do comércio de produtos com a capacidade explicativa e comunicativa do LLM Mistral 7B [3], deste modo, não gera apenas a classe do sentimento, mas também uma explicação textual, num ambiente 100% local.

A importância desta abordagem estabelecida acentua-se ao considerar os diversos domínios com textos constituídos por gírias e expressões idiomáticas muito comuns na língua portuguesa. No âmbito de avaliações de produtos, um utilizador elogia as especificações técnicas de um artigo, mas penaliza severamente o atraso na entrega. Perante este cenário de fronteira semântica, os classificadores matematicamente puros tem tendência em falhar ou em convergir para a classe neutra. Ao introduzir uma camada de *Explainable Artificial Intelligence* (XAI), o sistema não só oferece a decisão categórica tomada, como também instrui o utilizador sobre quais foram as palavras determinantes que tiveram preponderância na decisão do classificador, transformando a incerteza matemática em transparência interpretável.

Para além disso, a execução estritamente *offline* permite mitigar uma das principais barreiras da implementação de LLMs, o *data leakage* e a dependência de infraestruturas centralizadas. Ao coordenar o pipeline através do sistema *Ollama* e da biblioteca *Transformers* num ambiente de *hardware*, é demonstrada a viabilidade técnica para a implementação de soluções de PLN robustas, seguras e de baixo custo computacional.

Com o propósito de detalhar o desenvolvimento desta solução, este relatório encontra-se estruturado da seguinte forma:

- A Secção 3 aborda a arquitetura do sistema e a fundamentação das escolhas tecnológicas;
- A Secção 4.1 descreve o processo de engenharia de dados, a preparação do *corpus* e a estratégia do *fine-tuning* local realizado;
- A Secção 5 enaltece a avaliação quantitativa e qualitativa dos resultados obtidos através da interface interativa;
- Por fim, a Secção 6 compacta as principais conclusões e anota possíveis alterações e adições para trabalhos futuros nesta linha de investigação.

2. Recursos e Tecnologias Utilizados

2.1. Modelo de Classificação

O componente de classificação utiliza como ponto de partida o modelo **XLM-RoBERTa** `cardiffnlp/twitter-xlm-roberta-base-sentiment` [4]. Embora tenha uma performance excelente no domínio das redes sociais, mais concretamente no *Twitter*, o modelo sofre de discrepância de domínio (*domain shift*) quando aplicado a críticas de comércio. Para atenuar este problema, o modelo foi sujeito a um processo de *fine-tuning*, com a utilização da biblioteca `transformers`, através do ajuste dos seus pesos especificamente para a interpretação de avaliações de produtos em português.

A escolha da arquitetura XLM-RoBERTa justifica-se pela sua evolução direta face ao modelo original BERT [5]. Enquanto o BERT introduziu a revolução do processamento bidirecional baseado em mecanismos de atenção, o RoBERTa otimizou substancialmente esta arquitetura ao remover o objetivo de prever a próxima frase (*Next Sentence Prediction*) e ao introduzir o *masking* dinâmico de *tokens* (*Dynamic Masking*) ao longo da fase de treino. Na sua variante *Cross-Lingual Transfer* (XLM), o modelo adquire uma robustez acentuada na captação de nuances semânticas através de diversas línguas, garantindo desta forma uma base lexical forte e completa para processar todo tipo de expressões e gírias da língua portuguesa mesmo antes da adaptação ao domínio.

A Tabela 1 contempla a evolução arquitetural e dimensional entre a versão base do BERT e o modelo XLM-RoBERTa utilizado neste trabalho, evidenciando o aumento considerável da capacidade de representação vocabular e do volume de parâmetros.

Tabela 1

Comparação arquitetural e dimensional entre os modelos BERT base e XLM-RoBERTa.

Característica	BERT [5]	XLM-RoBERTa [4]
Vocabulário	30.000 <i>tokens</i> (apenas Inglês)	250.000 <i>tokens</i> (XLM)
Janela de Contexto	512 <i>tokens</i>	512 <i>tokens</i>
Dimensionalidade (<i>d</i>)	768	1024
Camadas (Layers)	12	24
Cabeças de Atenção	12 (Bidirecional)	16
Total de Parâmetros	≈ 100 Milhões	≈ 550 Milhões

2.2. Modelo Generativo Local

O componente de geração de explicações utiliza o modelo **Mistral 7B** [3], executado localmente através da plataforma *open-source* Ollama [6]. O Mistral 7B foi selecionado pelo seu notável desempenho com suporte para a língua portuguesa e por, na sua versão quantizada a 4-bits (≈ 4.1 GB), permitir inferência rápida em *hardware* local.

A aplicação de técnicas de quantização constitui um alicerce fundamental para a viabilidade de execução do sistema. Este processo reduz drasticamente a alocação de memória da placa gráfica (VRAM) necessária para alojar o modelo, sem provocar uma degradação perceptível na coerência lógica ou na fluência gramatical do texto gerado. Adicionalmente, o Mistral 7B integra otimizações nativas avançadas, como a *Grouped-Query Attention* (GQA), que aceleram substancialmente o processamento e a emissão de cada *token*. Ao encapsular este processo inteiramente no sistema isolado do Ollama, o sistema assegura que a explicação da decisão tomada (IA Explicável) ocorra num ambiente *offline*, garantindo assim que o conteúdo submetido nas avaliações nunca é transmitido para servidores na *cloud* (APIs de terceiros), respeitando a privacidade dos dados.

2.3. Interface Gráfica

A interface de utilizador foi construída com Gradio [7], com a disponibilização um ambiente *web* interativo onde o utilizador submete o texto e visualiza imediatamente a classificação, a distribuição probabilística das três classes e o parágrafo gerado pelo Mistral com a justificação da previsão feita.

O desenvolvimento da aplicação, centralizada no *file app.py*, foi estruturada através do paradigma de blocos do Gradio (*Gradio Blocks*), o que permitiu desenhar um *layout* atrativo e visualmente segmentado. A janela principal divide-se numa coluna de entrada (composta por uma caixa de texto simples para a inserção da crítica) e numa coluna de saída dos dados, organizada verticalmente para apresentar as componentes previstas e explicativas de forma hierárquica e clara.

Para além de simplificar a interação humana sendo uma aplicação bastante intuitiva, a interface atua como o cérebro que implementa a ideia por trás dos componentes presentes na *pipeline* híbrido. Assim que o evento de submissão é acionado pelo utilizador, o Gradio invoca a função principal do *script* *sentiment_analyzer.py*. Os resultados numéricos do nível um são mapeados dinamicamente para um componente gráfico de barras (*Label component*), através da renderização visual das percentagens de confiança. Em simultâneo, o fluxo de texto gerado pelo Mistral 7B através da API local do *Ollama* é capturado e injetado diretamente numa caixa de texto de leitura apenas, garantindo deste modo que o utilizador consiga consumir o veredicto estatístico e a sua fundamentação linguística numa única vista aglomerada.

2.4. Diferenciação Arquitetural: Modelos Encoder-Only vs. Decoder-Only

A evolução dos modelos baseados na arquitetura *Transformer* [1] bifurcou-se em duas abordagens fundamentais para o Processamento de Linguagem Natural (PLN), os modelos que se baseiam exclusivamente em (*Encoder-only*) e os modelos que se baseiam exclusivamente em (*Decoder-only*). A compreensão desta divergência estrutural é crucial para justificar a arquitetura híbrida proposta neste trabalho.

Por um lado, o XLM-RoBERTa [4] representa o paradigma *Encoder-only*. Esta arquitetura baseia-se no processamento bidirecional, onde todas as camadas do modelo retêm a capacidade de estabelecer conexões de atenção entre qualquer par de *tokens* na sequência de entrada, simultaneamente. O modelo é pré-treinado através de *Masked Language Modeling* (MLM), o que o torna ideal para tarefas de extração e classificação discriminativa. Ao analisar o texto de forma holística e contextual, o

componente *Encoder* maximiza o rigor estatístico na atribuição de probabilidades a classes discretas (Positivo, Negativo ou Neutro), contudo, carece de mecanismos eficientes para a geração de texto livre.

Relativamente ao *encoder*, a arquitetura do mesmo diferencia-se pela sua matriz de conexões densas de atenção (ilustrada na Figura 1) [8], onde a camada de cada nível processa e integra os *inputs* de todas as posições resultantes da camada anterior. Este mecanismo evidencia o procedimento da atenção bidirecional (*bidirectional self-attention*) que permite ao modelo capturar dependências semânticas independentemente da sua posição na frase.

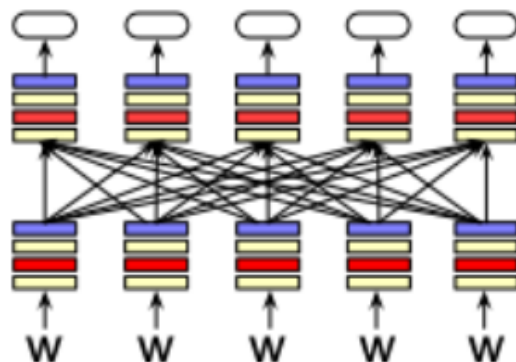


Figura 1: Representação do mecanismo de atenção bidirecional num modelo *Encoder-only*, demonstrando o fluxo simultâneo de informação entre todos os *tokens*.

Por outro lado, o Mistral 7B [3] assenta no âmbito de *Decoder-only*, sendo caracterizado pela geração de texto auto-regressiva. Ao contrário do *encoder*, o *decoder* utiliza máscaras causais (*causal self-attention*) que impedem os *tokens* atuais de conseguir aceder a posições futuras na sequência. O modelo prevê a próxima palavra exclusivamente com base no histórico de *tokens* anteriores e através de técnicas avançadas de atenção, como a *Grouped-Query Attention* (GQA). Esta abordagem confere-lhe uma capacidade generativa consistente, ideal para formular explicações complexas em linguagem natural. Contudo, fazer o mapeamento diretamente desta capacidade generativa enorme unicamente para prever uma classe discreta introduz latência desnecessária e uma menor consistência estatística quando comparado a um classificador dedicado.

Esta limitação de tempo é evidente na topologia da rede do *decoder* (Figura 2) [8], onde as conexões de atenção são *masked* assimetricamente. O processamento de um *token* restringe-se por completo à informação anterior, impedindo o modelo de olhar para a frente.

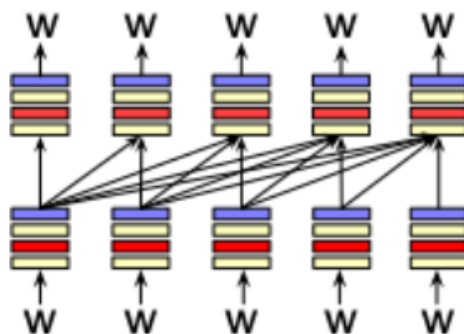


Figura 2: Mecanismo de atenção causal (mascarada) típico de modelos *Decoder-only*, onde o fluxo de informação é unidirecional para prevenir o *context leak* do futuro.

Como é possível observar na figura seguinte, a Figura 3 [9], o processo de pré-treino dos *Encoder-only* é centrado no *Masked Language Modeling* (MLM). Neste mecanismo, o modelo não atua de forma auto-regressiva como nos modelos *Decoder*, ao invés disso, a sequência de entrada é propositalmente alterada, substituindo uma percentagem dos *tokens* por uma máscara de oclusão (por exemplo, [MASK]). A camada final (*Masked LM Head*) tem como objetivo principal reconstruir e prever o *token* original oculto através do cálculo das probabilidades de distribuição (*logits*), por via de todo o contexto bidirecional, ou seja, ao processar em simultâneo a informação que ocorre tanto à esquerda como à direita da máscara. Este mecanismo força o modelo a desenvolver representações semânticas profundas de toda a frase.

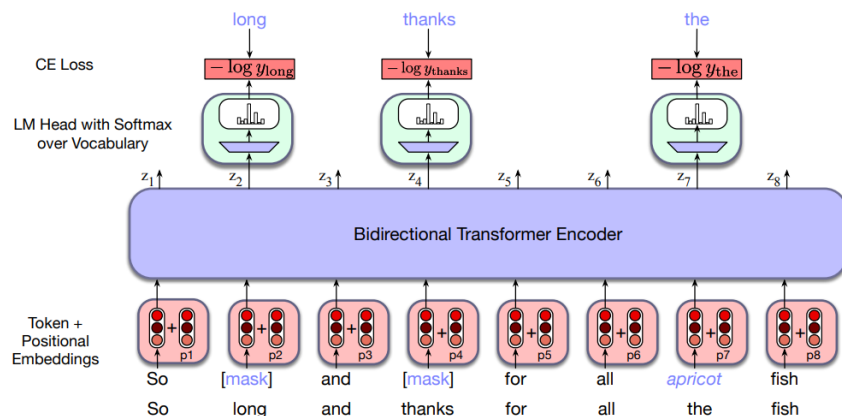


Figura 3: Estrutura do mecanismo de *Masked Language Modeling* (MLM) típico de um *Encoder*, demonstrando a previsão de *tokens* ocultos através do processamento do contexto bidirecional.

O sistema desenvolvido neste artigo tira partido do melhor dos dois mundos, atribui a classificação rápida e rigorosa ao *Encoder* (XLM-RoBERTa) e a explicabilidade interpretável humana ao *Decoder* (Mistral 7B), mitigando as limitações individuais de cada arquitetura.

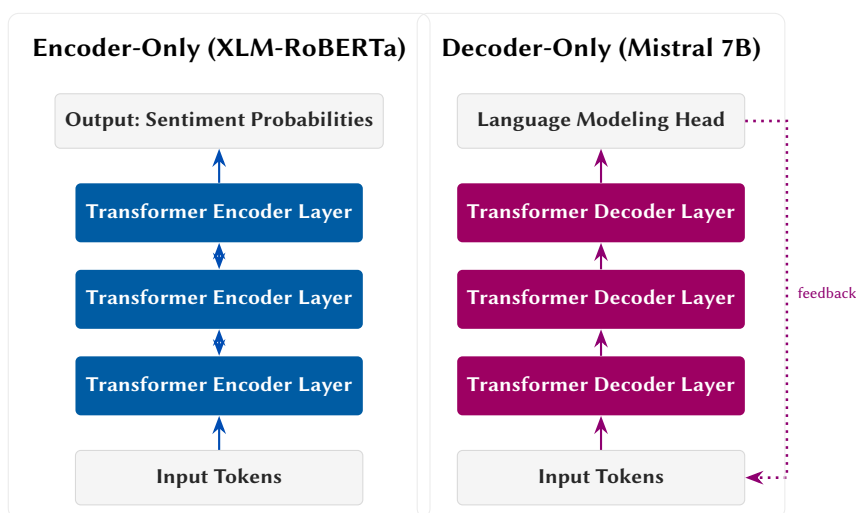


Figura 4: Comparação conceptual entre arquiteturas *Encoder-only* e *Decoder-only* utilizadas no sistema proposto.

3. Arquitetura do Sistema

O sistema proposto baseia-se numa arquitetura modular assente num *pipeline* sequencial de dois níveis independentes [10], concebidos para operar de forma estritamente local, garantindo o processamento *offline* e a total privacidade dos dados do utilizador. O fluxo macro de dados, bem como a interação entre os componentes e os módulos de *software* desenvolvidos, encontram-se ilustrados na Figura 5.

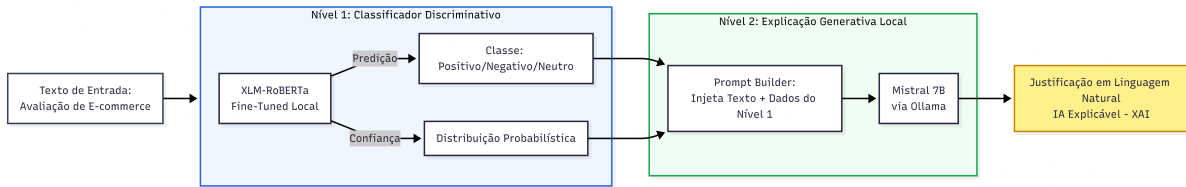


Figura 5: Arquitetura do sistema híbrido proposto, detalhando o fluxo da sequência entre o classificador discriminativo (nível um) e o gerador explicativo local (nível dois).

O funcionamento dinâmico desta solução implementada assenta na combinação de modelos robustos orientados para a extração estatística e para modelos generativos focados na interface humana, conforme detalhado nas subsecções seguintes.

3.1. Ingestão e Pré-processamento de Dados

O ponto de partida consiste na receção do *Texto de Entrada*, que realiza uma avaliação em linguagem natural proveniente do domínio de comércio ou do domínio das redes sociais em português. Antes de ser submetido à inferência de ambos modelos, o texto passa por uma camada leve de normalização, implementada nos scripts principais do sistema, que assegura a integridade dos *tokens* e previne falhas na codificação dos caracteres especiais comuns na língua portuguesa (como acentuação e cedilhas), preparando assim a string para a tokenização síncrona.

3.2. Classificador (XLM-RoBERTa)

Uma vez preparado o texto, este é inserido no *Classificador*, cujo o núcleo é constituído pelo modelo *encoder-only* XLM-RoBERTa, afinado localmente através de *fine-tuning* para mitigar as discrepâncias de domínios (*domain shift*). Esta etapa foca-se na eficiência computacional e na precisão matemática, ao executar quase instantaneamente para extrair dois outputs cruciais em paralelo:

- **Previsão Categórica:** A atribuição da *label* do sentimento, com as três classes possíveis Positivo, Negativo ou Neutro.
- **Distribuição Probabilística (Confiança):** Um vetor numérico que expressa o valor de probabilidade associado a cada uma das classes disponíveis (e.g., $P(\text{Negativo}) = 0.98$).

A separação explícita destes vetores de confiança é crucial, pois serve de fundação quantitativa para o nível seguinte do sistema.

Do ponto de vista do mecanismo, conforme exemplificado na topologia de rede da Figura 6, o processamento inicia com a fragmentação do texto de entrada em unidades mínimas de significado (*tokens*), introduzindo em paralelo o *token* especial de controlo [CLS] na primeira posição da sequência. Estes elementos são convertidos na camada de (*embeddings*, mapeados como $E_{[\text{CLS}]}, E_1, E_2 \dots E_N$) e submetidos às camadas profundas do XLM-RoBERTa. Como é possível observar nas conexões densas internas, o modelo processa o contexto de forma inteiramente bidirecional, permitindo que cada posição influencie e seja influenciada pelas restantes. Para efetuar a classificação final, os estados ocultos intermédios dos *tokens* textuais ($T_1 \dots T_N$) são descartados, e extrai-se exclusivamente e apenas o vetor de contexto do primeiro bloco, denotado como C . Este vetor é encaminhado diretamente para uma camada de ativação *Softmax*, responsável por mapear os scores brutos (*logits*) nas probabilidades discretas de sentimento (*Class Labels*) que são indispensáveis para o pipeline.

3.3. Explicação Generativa Local (Mistral 7B)

O **Mistral** é acionado de forma estritamente sequencial, recebendo o *payload* de dados gerado na etapa anterior. O funcionamento deste nível divide-se em dois blocos:

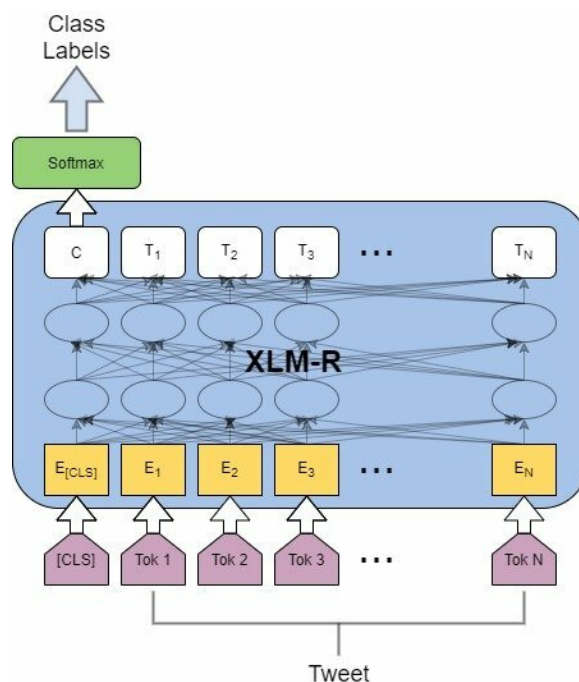


Figura 6: Fluxo de processamento e topologia interna de classificação do XLM-RoBERTa, destacando o embedding bidirecional e o isolamento do estado oculto C para a camada *Softmax* (adaptado de Ranasinghe e Zampieri [11, 8]).

3.3.1. Módulo Prompt Builder

O *Prompt Builder* atua como uma ponte de tradução entre a lógica matemática e a cognitiva. Este componente recebe o texto original da crítica, a classe que o modelo previu e as respectivas probabilidades de confiança do XLM-RoBERTa. Através de algoritmos de interpolação de *strings*, o método reconstrói e formata um *prompt* estruturado de *closed context*. O *prompt* é delineado com indicações sistêmicas claras, delimitando o sentido de atuação do LLM para evitar possíveis alucinações.

3.3.2. Inferência Local via Mistral 7B e Ollama

O bloco de contexto estruturado é enviado para o *decoder* Mistral 7B, instanciado localmente através do sistema *open-source* do *Ollama*. Ao utilizar uma versão otimizada com quantização de 4-bits, o modelo consome o *prompt* e processa as instruções textuais fornecidas. O Mistral 7B é instruído especificamente para identificar as expressões chave ou determinantes linguísticos que justificam a classificação estatística do nível um, através da formulação de uma justificação em Linguagem Natural encapsulada em português europeu.

A viabilidade da computação local deste volume elevado de parâmetros é sustentada pelas inovações estruturais introduzidas na parte interna do Mistral 7B [3]. Como consta no diagrama de blocos da Figura 7, o fluxo de dados estende-se por uma estrutura repetitiva de $N \times$ camadas de *Transformers*. O núcleo deste processamento habita na camada de *Self-Attention* otimizada nativamente com *Grouped-Query Attention* (GQA) e *Sliding Window Attention* (SWA), que executam sobre projeções rotativas de *embeddings* posicionais (*Rotary Position Embeddings* (RoPE)). Aditivamente, a substituição da *LayerNorm* tradicional por blocos de pré-normalização baseados em *Root Mean Square Normalization* (RMS Norm) acelera de forma significativa o tempo de cálculo por cada camada, garantindo a estabilidade e a rapidez da inferência em *hardware* local.

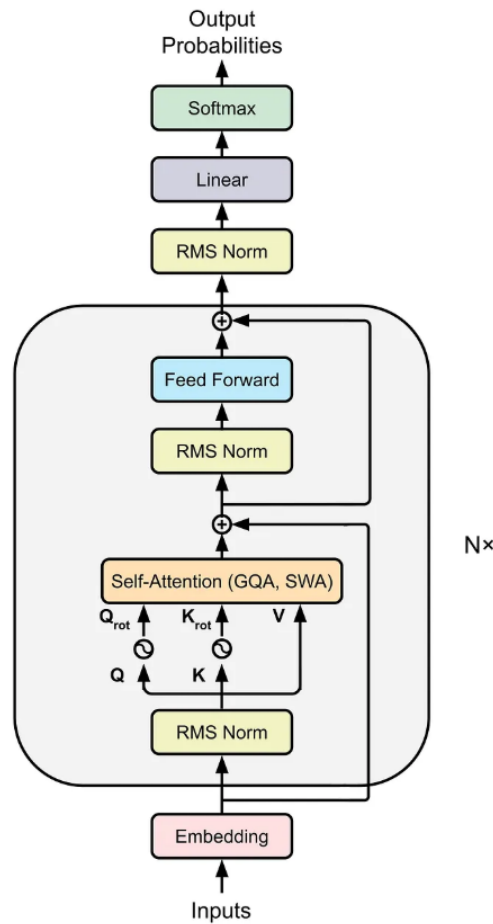


Figura 7: Diagrama detalhado da arquitetura interna de blocos do Mistral 7B, evidenciando os componentes de normalização RMS e o módulo de atenção GQA/SWA adaptado de [3, 9].

Desta forma, perante cenários de elevada ambiguidade semântica, o sistema não se limita a fornecer uma classificação para a instância atual, converte os dados numéricos de carácter *black box* em explicações inteligíveis de *Explainable Artificial Intelligence* (XAI), elevando de forma significativa a fiabilidade global do sistema.

4. Implementação e Adaptação de Domínio

O sistema está organizado numa arquitetura modular em *Python* [12], garantindo a separação de responsabilidades. Para colmatar a falha de precisão do modelo base, implementou-se um processo rígido de preparação de dados e treino.

4.1. Preparação do Dataset

Para realizar a adaptação de domínio do *classifier*, o *script* `prepare_dataset.py` efetuou a extração de uma amostra a partir do *corpus* público *B2W-Reviews* [13]. Este *corpus* é constituído por avaliações reais de clientes no contexto do comércio eletrónico, contendo textos ricos e carregados de expressões idiomáticas, críticas a produtos, elogios e comentários de insatisfação típicas neste âmbito digital em língua portuguesa.

O processo de amostragem e divisão (*split*) dos dados foi desenhado da seguinte forma:

- **Conjunto de Avaliação (Test Set):** Foram extraídas 300 avaliações do *corpus* original. Este conjunto foi perfeitamente equilibrado *à priori*, contendo exatamente 100 exemplos pertencentes

à classe Negativo, 100 da classe Neutra e mais 100 da classe Positiva. Este conjunto funciona como um ambiente fechado e nunca foi exposto ao modelo durante a fase de otimização, respeitando assim a regra básica da área de *Machine Learning*.

- **Conjunto de Treino (*Training Set*):** Foram extraídos 3000 exemplos distintos do mesmo *corpus* original para a atualização dos pesos sinápticos do XLM-RoBERTa.

Crucialmente, para mitigar o fenómeno de *data leakage* (fuga de dados), que iria enviesar os resultados de *accuracy* ao testar o modelo em frases previamente conhecidas, o *pipeline* do script garante que existe uma exclusão mútua absoluta. Nenhuma das 300 frases que constituem o ficheiro `dataset_avaliação.csv` está presente ou foi amostrada para o lote das 3000 frases utilizadas no *fine-tuning*, garantindo a plenitude científica da avaliação quantitativa de 78.0% de *accuracy* geral obtida.

4.2. Fine-Tuning

O *file* `train_model.py` encarrega-se do ajuste de pesos do modelo XLM-RoBERTa. O treino foi executado ao longo de quatro épocas, recorrendo à classe *Trainer* da Hugging Face, utilizando um *Data Collator* dinâmico para otimização de memória e um mecanismo de *Early Stopping* para abortar o treino em caso de *overfitting*.

Este processo de otimização estendeu-se por um máximo de 4 épocas pré-definidas, recorrendo a uma abstração de alto nível atribuída pela classe *Trainer* da biblioteca *Hugging Face*, integrada nativamente com o *PyTorch*. Para garantir que a performance por parte deste sistema fosse então melhorada através deste ajuste, os principais hiperparâmetros definidos no *TrainingArguments* encontram-se sistematizados na Tabela 2.

Tabela 2

Hiperparâmetros configurados para o *fine-tuning* local do XLM-RoBERTa.

Hiperparâmetro	Valor Configurado
Taxa de Aprendizagem (<i>Learning Rate</i>)	2×10^{-5}
Dimensão do Lote de Treino (<i>Train Batch Size</i>)	8
Dimensão do Lote de Avaliação (<i>Eval Batch Size</i>)	8
Otimizador	AdamW
Decaimento de Pesos (<i>Weight Decay</i>)	0.01
Métrica de Monitorização	<i>Validation Loss</i>
<i>Patience</i> (<i>Early Stopping</i>)	2 épocas

A execução do treino num ambiente local impôs restrições de memória de vídeo (VRAM). Para minimizar possíveis erros de *Out Of Memory* (OOM) sem prejudicar a estabilidade, foram implementadas duas estratégias fundamentais na engenharia do *pipeline*:

1. ***Data Collator*:** Em vez de aplicar um preenchimento (*padding*) estático com base no comprimento máximo global suportado pelo modelo (512 *tokens*), o *DataCollatorWithPadding* ajusta dinamicamente a dimensão das matrizes de entrada ao comprimento do maior elemento presente em cada *mini-batch* específico. Esta técnica reduz drasticamente a computação de *tokens* nulos desnecessários, otimizando a alocação de memória na GPU a cada iteração.
2. **Mecanismo de *Early Stopping*:** Para prevenir a existência de *overfitting* e garantir a eficiência temporal do treino, assegurou-se a integração de um *callback* de paragem antecipada, a então denominada de *Early Stopping* [14]. O algoritmo foca-se em monitorizar o custo no conjunto de validação ($\mathcal{L}_{val}$) ao fim de cada época de avaliação, sendo o treino logo abortado imediatamente que se verifique:

$$\mathcal{L}_{val}^{(t)} \geq \mathcal{L}_{val}^{(t-1)} \quad (1)$$

durante um intervalo de tolerância (*patience*) equivalente a duas épocas completas, desfazendo o estado do modelo para os melhores pesos registados (`load_best_model_at_end`).

5. Experiências Realizadas e Resultados Obtidos

5.1. Avaliação Quantitativa (Classificador)

O sistema foi avaliado num *dataset* de teste à parte de 300 amostras. O modelo original de base (treinado em *tweets*) apresentava uma acurácia (*accuracy*) de 64.3% apenas, demonstrando uma enorme dificuldade com a classe Neutra.

Após o processo de *fine-tuning* implementado neste projeto, o classificador do nosso sistema obteve uma melhoria considerável, atingindo uma *accuracy* de 78.0%. A Tabela 3 detalha as métricas finais.

A desproporção do desempenho entre o estado inicial do classificador e o comportamento atingido após o ajuste de pesos está declarada visualmente na Figura 8.

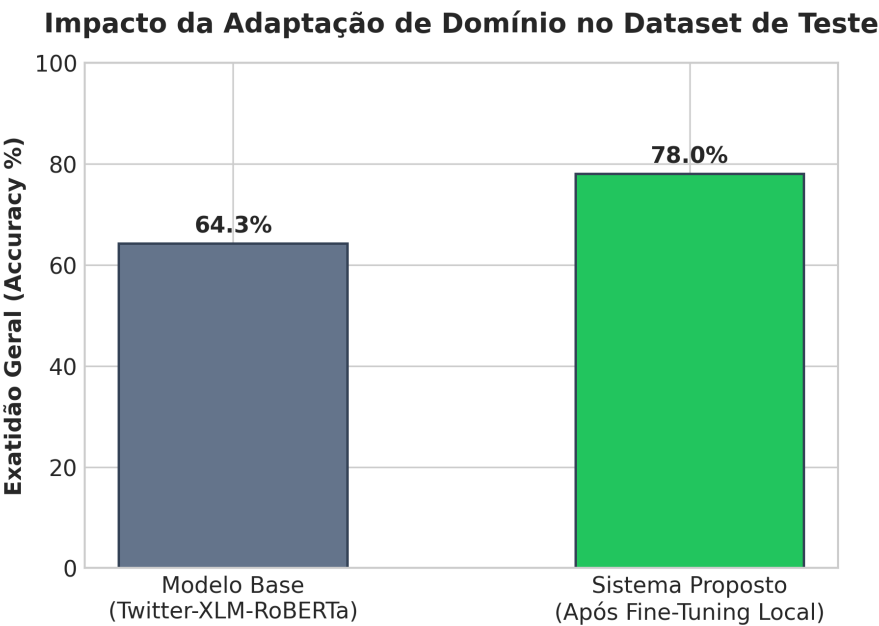


Figura 8: Gráfico comparativo da Acurácia Geral (Accuracy) no conjunto de teste antes e após o processo de *fine-tuning*.

Tabela 3

Resultados de avaliação do XLM-RoBERTa após *fine-tuning* no conjunto de teste (300 amostras).

Classe	Precision	Recall	F1-score	Suporte
Negativo	0.79	0.88	0.83	100
Neutro	0.72	0.63	0.67	100
Positivo	0.82	0.83	0.83	100
Exatidão Geral	78.0%			300

A fim de complementar a análise da Tabela 3, a Figura 9 decompõe o comportamento do modelo através de um gráfico de barras agrupadas, permitindo inspecionar o equilíbrio entre a precisão e a sensibilidade obtidas em cada uma das categorias de sentimento estudadas.

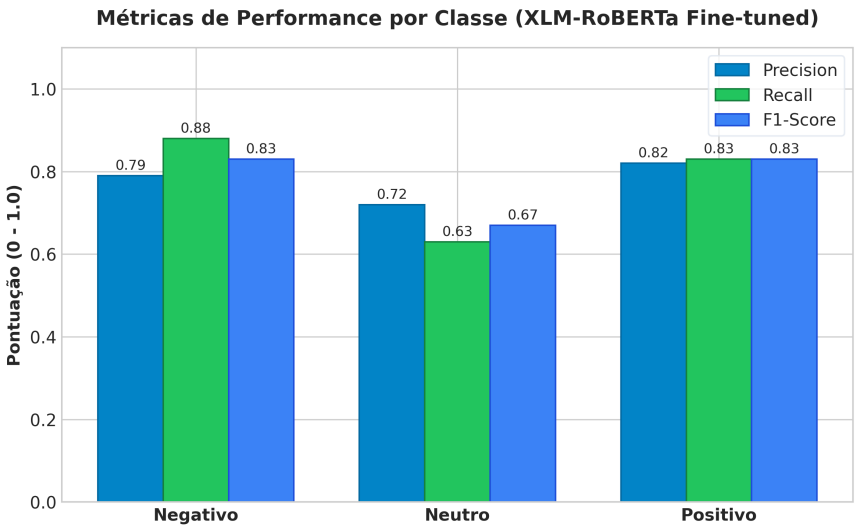


Figura 9: Distribuição detalhada das métricas de Precision, Recall e F1-Score por classe obtidas pelo classificador XLM-RoBERTa.

Apesar da alta precisão nas classes polares (F1 de 0.83 para Positivo e Negativo), a classe Neutra continua a representar o maior desafio analítico (F1 de 0.67). Esta estagnação deve-se à ambiguidade humana presente nos textos da sua autoria avaliações de três estrelas ("Neutras") raramente expressam apatia, revelando ao invés disso sentimentos mistos numa mesma frase (ex., elogio ao produto, mas crítica à entrega).

5.2. Avaliação Qualitativa (IA Explicável)

É precisamente perante a ambiguidade da classe Neutra que o nível dois do sistema revela o seu valor. Quando o classificador probabilístico é forçado a rotular um sentimento misto, o Mistral 7B descodifica essa decisão, mitigando a percepção de erro por parte do utilizador. O excerto abaixo ilustra esta mitigação perante uma frase avaliada como ambígua e ainda um outro exemplo de uma gíria portuguesa:

Listing 1: Comportamento do sistema perante ambiguidade de domínio.

Texto: "O produto e espetacularmente horrivel."
Sentimento Detetado: Negativo (Confianca: 98\%)
Explicacao gerada pelo Mistral 7B:
1. O sentimento foi classificado como Negativo pois a descricao do produto utiliza palavras tais como "horrivel", indicando uma opiniao negativa.
2. As palavras ou expressoes chave que mais contribuíram para esta classificacao sao "espetacularmente" e "horrivel". A primeira, usada no contexto, enfatiza a intensidade da desaprovacao, enquanto a segunda indica uma opiniao negativa sobre o produto.

Texto: "Nao metas a foice em seara alheia."
Sentimento Detetado: Negativo (Confianca: 97.4\%)
1. O sentimento foi classificado como negativo devido a presenca da palavra "foice" (forceps), que e usada para cortar coisas ou sofrer ferimentos, e da frase "metas a foice em seara alheia",

- que sugere um ato agressivo ou invasivo contra uma propriedade ou território de outra pessoa.
2. As palavras ou expressões chave que mais contribuíram para a classificação negativa do sentimento são: "foice" e "seara alheia".

Através desta arquitetura, um erro probabilístico é transformado numa diferença de perspectiva interpretável, aumentando substancialmente a fiabilidade e credibilidade do sistema.

Para verificar e validar a versatilidade da interface construída e consolidar a análise qualitativa do sistema local, foram registados cenários de teste correspondentes às três orientações clássicas de feedback de utilizadores.

Na Figura 10, observa-se a resposta do pipeline perante uma avaliação de carácter benéfico. O nível um regista uma certa matemática expressiva (92.8%) na classe Positivo, e o Mistral 7B consegue extrair com clareza os determinantes semânticos de satisfação.

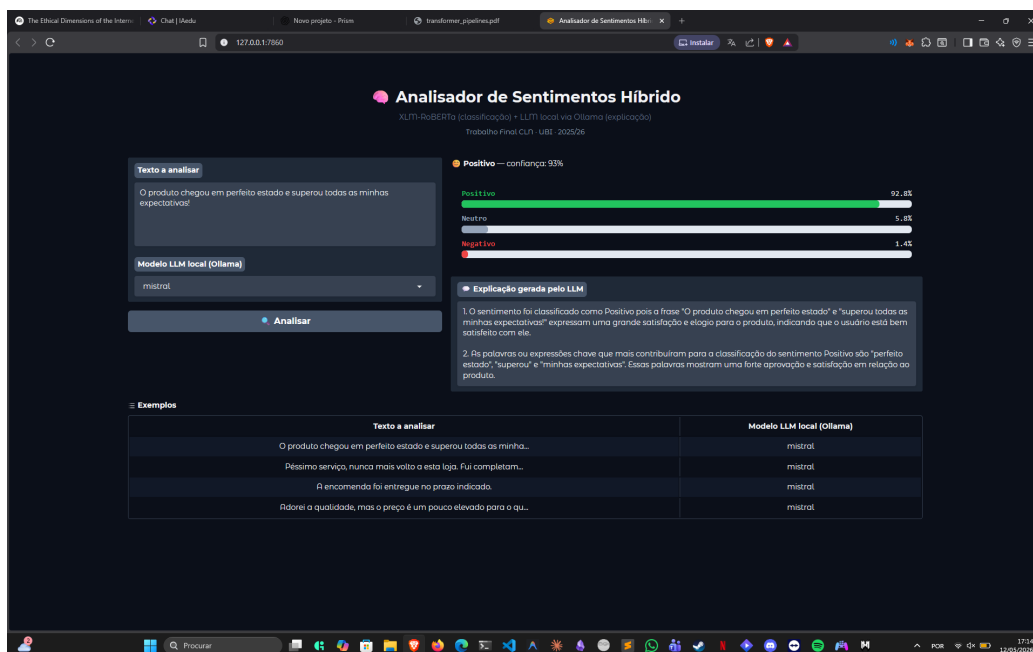


Figura 10: Interface gráfica do sistema contendo uma classificação com elevada confiança na classe Positiva e respetiva fundamentação do LLM.

Em contrapartida, a Figura 11 demonstra o comportamento do sistema perante uma crítica explícita de insatisfação. O classificador classifica com elevada confiança a polaridade Negativa (95.8%), enquanto o bloco explicativo esclarece com sucesso o comportamento por trás do texto do utilizador de quebrar o vínculo com o estabelecimento.

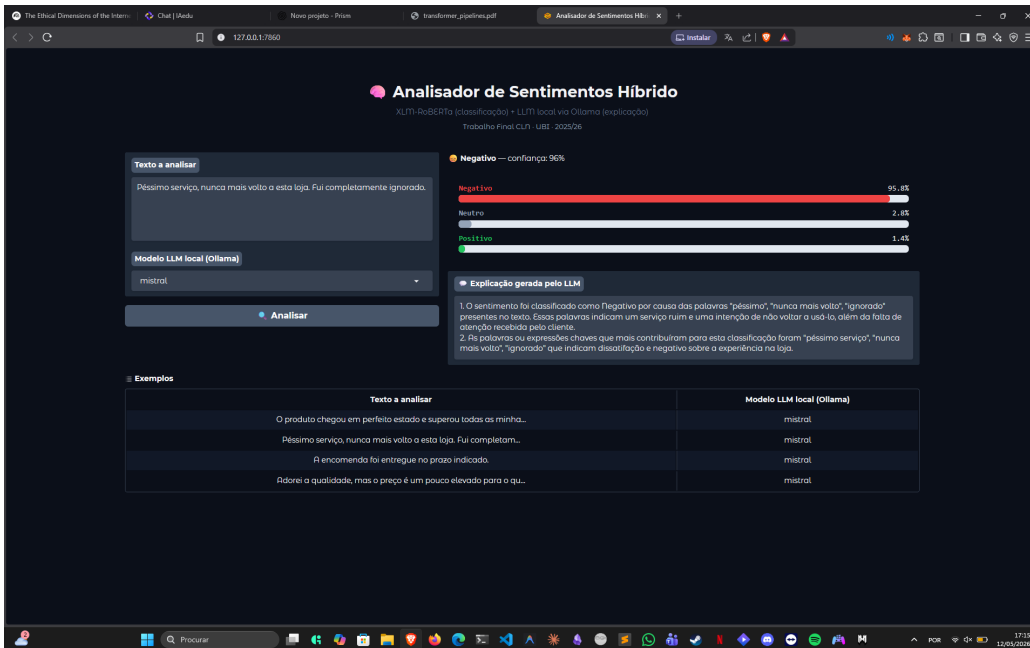


Figura 11: Demonstração da interface gráfica durante a submissão de uma crítica classificada com 95.8% de polaridade Negativa.

Por fim, a Figura 12 exibe um cenário crucial de um limiar interpretativo, caracterizado por sentimentos mistos numa declaração textual (apreço pela qualidade técnica versus objeção sobre o preço de mercado). O XLM-RoBERTa, embora atribua prioridade à classe Negativa com 59.8% devido ao peso léxico das palavras desse cariz, divide a incerteza estatística com a classe Neutra (28.9%). É perante este limite matemático que o nível dois consegue minimizar a ambiguidade do sistema, expondo ao utilizador a raiz do raciocínio e justificando de forma equilibrada o peso relativo do valor obtido.

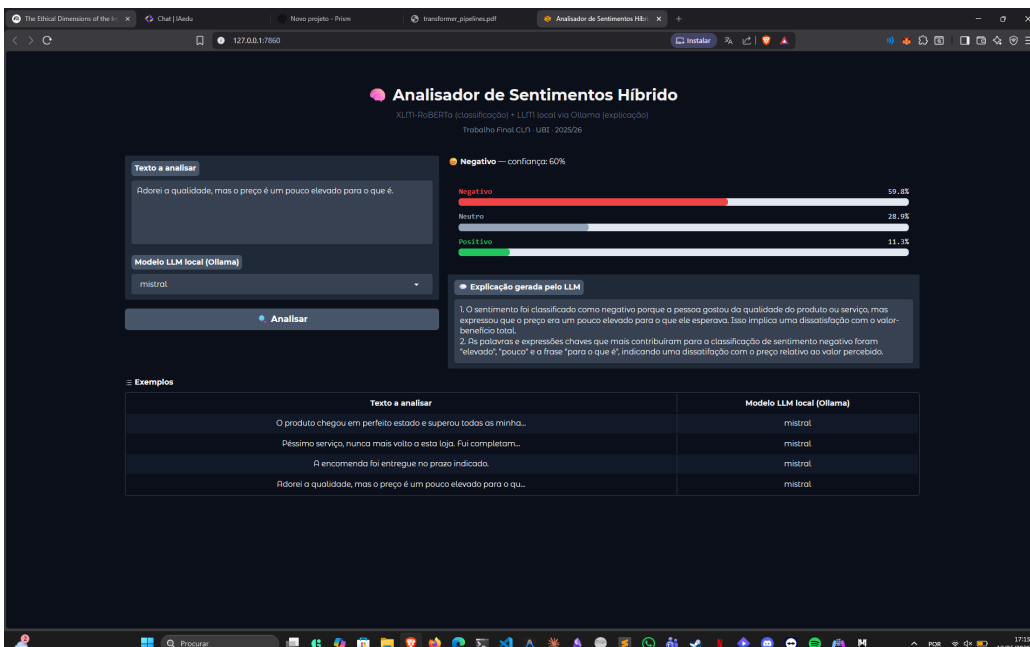


Figura 12: Performance do sistema perante uma frase de sentimentos mistos com distribuição probabilística partilhada.

6. Conclusão

Este trabalho demonstrou a viabilidade da criação de uma arquitetura de Processamento de Linguagem Natural híbrida, capaz de operar de forma estritamente local, sem comprometer a eficácia do sistema.

O *fine-tuning* local do modelo discriminativo provou ser essencial para adaptar o sistema ao domínio do comércio eletrônico, elevando a precisão global de 64.3% para 78.0%. Perante o limite matemático imposto pela ambiguidade semântica (verificado sobretudo na classe Neutra), a integração do LLM Mistral 7B constituiu uma resposta eficaz no âmbito da IA Explicável (XAI). O sistema final não procura criar um classificador infalível, mas sim um classificador transparente, em que o raciocínio subjacente a cada decisão é justificado ao utilizador em tempo real e em linguagem natural.

As contribuições deste relatório são abrangidas para além do aumento da performance em termos de acurácia, provando que as restrições de *hardware* não impedem o desenvolvimento de sistemas mais complexos de PLN. A descentralização das ferramentas de inteligência artificial e a sua transição para ambientes *offline* abrem precedentes extremamente cruciais para a segurança dos dados em auditorias de consumo, protegendo deste modo a privacidade individual e coletiva sem abdicar da capacidade de síntese cognitiva oferecida pelos modelos de grande escala da última geração.

Como perspetiva para trabalho futuro, idealiza-se a evolução deste pipeline aqui instanciado através da implementação de uma arquitetura de Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* (RAG)) [15]. Esta extensão seria viabilizada através do recorrimto à *framework LangChain* [16], permitindo assim conectar de forma dinâmica o Mistral 7B a uma base de dados vetorial local carregada com todo o tipo conteúdo indispensável para as tarefas do sistema, como por um exemplo no âmbito do comércio, os documentos presentes na database poderiam conter informação relativa ao histórico completo de inventários, a políticas de devolução e a catálogos de produtos da empresa.

Desta forma, o nível dois do sistema não se limitará apenas a analisar a semântica da frase proveniente do cliente, mas será capacitado para contextualizar a sua justificação textual cruzando-a com dados reais do negócio em tempo real. Esta integração permitirá fornecer respostas de suporte explicativo ainda mais ricas, personalizadas e acionáveis, estando o sistema muito mais informado e enquadrado com a atualidade, expandindo o âmbito do sistema de um assistente interpretável para uma ferramenta proativa de apoio à tomada de decisão.

Acknowledgments

Agradecer ao docente da unidade curricular de Computação em Linguagem Natural professor Doutor João Paulo Cordeiro, pela orientação, pelos materiais disponibilizados ao longo do semestre e pelo ensinamento dos conteúdos indispensáveis para a realização deste projeto.

Declaration on Generative AI

Durante a preparação deste trabalho, foi utilizado o Gemini para apoio à implementação do código e o ChatGPT para apoio à escrita e estruturação deste relatório. Para a arquitetura do sistema foi utilizado também a inteligência artificial do Mermaid Chart para prevalecer a riqueza e entendimento da arquitetura deste sistema implementado. Após a utilização destas ferramentas, o conteúdo foi revisto cuidadosamente e editado sempre que necessário, assumindo desta forma total responsabilidade pelo conteúdo final da publicação.

Referências

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in neural information processing systems, volume 30, 2017.
- [2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, arXiv preprint arXiv:1907.11692 (2019).
- [3] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al., Mistral 7b, arXiv preprint arXiv:2310.06825 (2023).
- [4] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, V. Stoyanov, Unsupervised cross-lingual representation learning at scale, arXiv preprint arXiv:1911.02116 (2019).
- [5] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805 (2018).
- [6] Ollama, Ollama: Get up and running with large language models locally, <https://ollama.com>, 2023. Acedido em: 13 de junho de 2026.
- [7] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, J. Zou, Gradio: Hassle-free sharing and testing of machine learning models in the wild, arXiv preprint arXiv:1906.02569 (2019).
- [8] U. Departamento de Informática, Computação em Linguagem Natural – Teórica 07: Modelos de Linguagem e Arquiteturas Encoder-Only, Universidade da Beira Interior, 2026. URL: https://moodle.ubi.pt/pluginfile.php/804155/mod_resource/content/7/CLN2026-T07.pdf, disponibilizado na plataforma Moodle (Acedido em: 13 de junho de 2026).
- [9] U. Departamento de Informática, Computação em Linguagem Natural – Teórica 09: Arquiteturas Generativas e Modelos Decoder-Only, Universidade da Beira Interior, 2026. URL: https://moodle.ubi.pt/pluginfile.php/806394/mod_resource/content/9/CLN2026-T09.pdf, disponibilizado na plataforma Moodle (Acedido em: 13 de junho de 2026).
- [10] K. Sveidqvist, The Mermaid Team, Mermaid: A markdown-ish syntax for generating flowcharts, sequence diagrams, and more, <https://mermaid.js.org>, 2025. Software Version 11.x, Acedido em: 13 de junho de 2026.
- [11] T. Ranasinghe, M. Zampieri, Faking sandy: Characterizing and detecting fake news on social media, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, 2020, pp. 4039–4053.
- [12] G. van Rossum, Python Software Foundation, Python language reference, version 3.x, <https://www.python.org>, 2026. Disponível em python.org, Acedido em: 13 de junho de 2026.
- [13] T. Filgueiras, Grupo B2W Digital, et al., B2w-reviews01: An open corpus for sentiment analysis in portuguese e-commerce, arXiv preprint arXiv:1801.00001 (Dataset Shared Publicly) (2018). URL: <https://github.com/americanas-tech/b2w-reviews01>.
- [14] L. Prechelt, Early stopping – but when?, in: Neural Networks: Tricks of the Trade, Springer, 2012, pp. 53–67.
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, M. Lewis, S. Riedel, T. Rocktäschel, D. Yogatama, G. Welbl, S. Riedel, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: Advances in Neural Information Processing Systems (NeurIPS), volume 33, 2020, pp. 9459–9474.
- [16] H. Chase, Langchain: Building applications with llms through composability, <https://github.com/langchain-ai/langchain>, 2022. Acedido em: 13 de junho de 2026.

A. Estrutura de Ficheiros do Projeto

O código-fonte submetido em anexo está organizado da seguinte forma:

```
projeto /
  app.py          # Interface web (Gradio)
  sentiment_analyzer.py  # Pipeline com RoBERTa + Mistral
```

<code>prepare_dataset.py</code>	# Pre-processamento e split do B2W
<code>train_model.py</code>	# Script de fine-tuning (PyTorch)
<code>evaluate.py</code>	# Script de avaliacao e metricas
<code>dataset_avaliacao.csv</code>	# Conjunto de teste isolado (300 lines)
<code>visualize.py</code>	# Graficos aqui presentes no relatorio